



Подсистема CRC Python IDE для анализа и ассистирования модификаций кода

Кузнецов Илья Александрович

Выпускная квалификационная работа

Уровень образования: магистратура

Направление *09.04.04 «Программная инженерия»*

Основная образовательная программа *ВМ.5666.2023 «Программная инженерия»*

Научный руководитель:

профессор кафедры системного программирования, д. т. н. Кознов Д. В.

Консультант:

ведущий инженер ключевых проектов ООО «Техкомпания Хуавэй», Тропин Н. В.

Рецензент:

ведущий инженер ключевых проектов ООО «Техкомпания Хуавэй», к. ф.-м. н. Волков М. В.

Санкт-Петербургский государственный университет

Актуальность и проблема

- Python широко используется в серверной разработке, анализе данных и ИИ-системах
- Современные Python-проекты требуют инструментов анализа, исправления и преобразования кода, а также контекстного ассистирования
- По данным Stack Overflow Developer Survey, 84% разработчиков используют или планируют использовать ИИ-инструменты
- ИИ-агенты повышают требования к точности семантической информации о коде
- CRC Python IDE нуждается в единой подсистеме анализа, модификации кода и информационного ассистирования

Цель и задачи

Цель

Проектирование и реализация подсистемы для среды разработки CRC Python IDE, которая должна предоставлять инструменты (Features) для анализа и ассистирования модификаций Python-кода

Задачи

- Провести обзор инструментов для анализа и ассистирования модификаций кода в существующих средствах разработки для языка Python
- Выполнить проектирование подсистемы, определив группы инструментов, необходимых пользователям, а также улучшения модели кода CRC Python IDE, необходимые для реализации этих инструментов
- Выполнить реализацию спроектированной подсистемы и интегрировать ее в CRC Python IDE
- Провести всестороннее тестирование разработанной подсистемы, модернизировав существующую тестовую инфраструктуру в проекте CRC Python IDE
- Провести апробацию и доработку подсистемы на основе обратной связи от пользователей CRC Python IDE

Анализ существующих решений

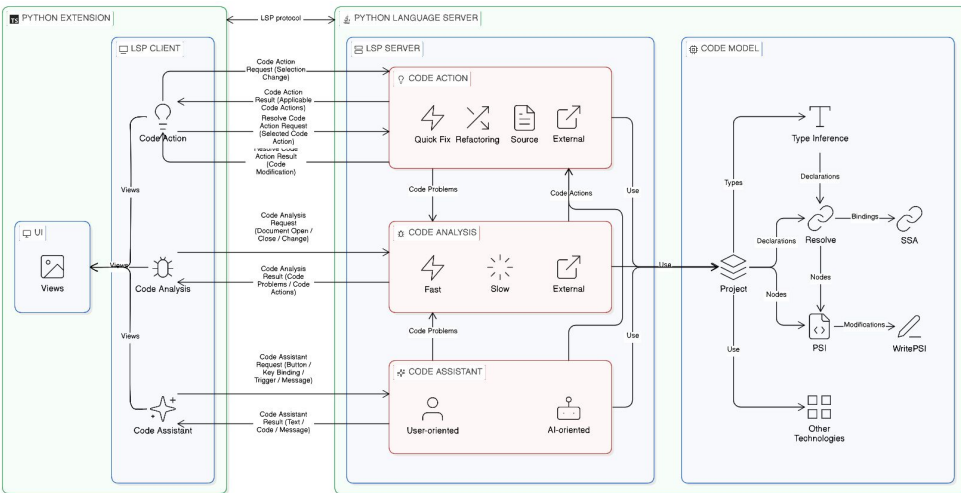
- Статические анализаторы — Мypy, Pyright, Pyrefly
 - Анализ ссылок и типов
 - Диагностика ошибок
 - Ограниченные возможности модификации кода
- Среды разработки (IDE) — PyCharm, VSCode + Pylance, CRC Python IDE
 - Анализ и навигация
 - Исправления и рефакторинг
 - Контекстные подсказки
- Системы на основе ИИ-агентов (ADE) — IntelliJ + AI Assistant, VSCode + Copilot, Cursor
 - Диалоговые сценарии
 - Агентные модификации
 - Зависимость от качества контекста

Сравнение существующих решений

Класс решений	Особенности	Ограничения
Статические анализаторы	Точная диагностика	Ограниченная поддержка модификаций
Среды разработки (IDE)	Интеграция анализа, исправлений и навигации	Ограниченная автономность
Системы на основе ИИ-агентов (ADE)	Агентные и диалоговые сценарии	Зависимость от качества контекста

- Требуется единая подсистема, объединяющая
 - Анализ кода
 - Модификацию кода
 - Исправление
 - Рефакторинг
 - Информационное ассистирование
 - Пользователей
 - ИИ-агентов

Архитектура



Улучшения модели кода

- PSI¹
 - Унифицированный доступ к синтаксическим и семантическим элементам программы
 - Стабилизация операций модификации дерева программы
- Resolve²
 - Конфигурируемое разрешение ссылок
 - Поддержка различных контекстов анализа
- Type Inference^{3, 4}
 - Расширение интерфейса вывода типов
 - Структурированное представление типов и их атрибутов

1 — Божнюк А. // Система модификации структуры исходного кода для интегрированных сред разработки // 2023 // СПбГУ

2 — Степанов С. // Потокочувствительный анализ программ на языке Python // 2023 // ИТМО

3 — Мирошников В. // Статический вывод типов для языка Python в интегрированной среде разработки // 2023 // СПбГУ

4 — Мирошников В. // Инкрементальный статический вывод типов для языка Python // 2025 // СПбГУ

Реализация: анализ и модификация кода

- Статический анализ кода
 - 40 инструментов
 - Syntax Correctness, Reference Resolve, Expected Type, Unreachable Code и др.
- Исправление кода
 - 12 инструментов
 - Install Package, Import Reference, Create Reference, Add Qualifier и др.
- Рефакторинг кода
 - 8 инструментов
 - Extract, Inline, Rename, Optimize Imports и др.

```
1 from non_existent_module import some_function
```

⊗ reference_resolve.py 1 of 1 problem

Unresolved import reference 'non_existent_module' Python(importReferenceResolve)

Реализация: информационные ассистенты

- Информационные ассистенты для пользователей
 - 3 инструмента
 - Hover, Signature Help, Inlay Hints
- Информационные ассистенты для ИИ-агентов
 - 4 инструмента
 - Problems MCP, Hover MCP, Signature Help MCP, Rename MCP

```
1  
2  
3  
4  
5 print("message", flush=True)
```

```
*values: tuple[object], sep: str | None = " ", end:  
^ str | None = "\n", file: _SupportsWriteAndFlush[str]  
2/2 | None = None, flush: bool  
v
```

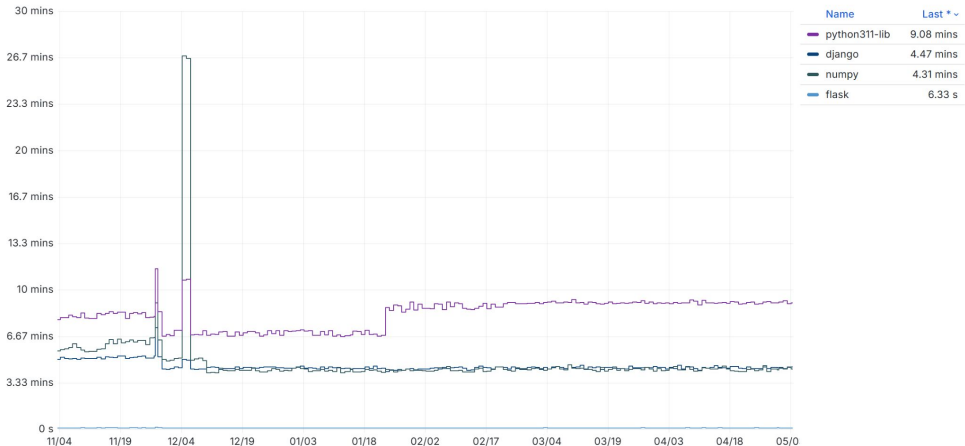
Тестирование

- Модернизация тестовой инфраструктуры
 - Конфигурируемые тестовые проекты
 - Фиксация ожидаемых результатов
 - Поддержка позиций курсора и диапазонов
- Модульное тестирование
 - 3000 тестовых сценариев
 - Покрытие всех подгрупп инструментов
 - Пограничные случаи
- Регрессионное тестирование
 - 5 бенчмарков
 - 30 выборочных Python-проектов
 - Контроль деградаций
- Комплексное тестирование
 - Автотесты на основе фреймворка Selenium
 - Воспроизводимые пользовательские сценарии
 - Проверка взаимодействия редактора, языкового сервера и UI

Мониторинг

- База данных Victoria Metrics и система визуализации Grafana
- Метрики количественных результатов, потребляемой памяти, времени анализов, скорости модификаций и латентности ассистентов

Project All Inspections Time



Апробация

- Апробация проведена на основе отзывов пользователей CRC Python IDE
- Реализовано 3 новых инструмента
 - Version Compatibility, Add Superclass Constructor Call, Optimize Imports
- Исправлено 10 проблем
 - Ложно-положительные/отрицательные срабатывания в анализах кода, некорректные/неэквивалентные модификации кода и нерелевантные подсказки ассистентов

Заключение

Результаты

- Рассмотрены инструменты для анализа и ассистирования модификаций кода в статических анализаторах Мypy, Pyright и Pyrefly, а также в средах разработки (IDE) PyCharm, VSCode и CRC Python IDE и, наконец, в системах на основе ИИ-агентов (ADE) AI Assistant, Copilot и Cursor; определены ключевые особенности этих решений и соответствующие им запросы Python-разработчиков
- Спроектирована подсистема среды разработки, включающая 3 следующие группы инструментов: (i) статический анализ, (ii) исправление и рефакторинг кода, а также (iii) информационные ассистенты для пользователей и ИИ-агентов; спроектированы необходимые улучшения модели кода CRC Python IDE: расширение и стабилизация синтаксического и семантического представления программы (PSI), уточнение и конфигурирование возможностей по разрешению ссылок (Resolve), доработка интерфейса вывода типов (Type Inference)

Заключение

- Выполнена реализация спроектированной подсистемы
 - реализованы улучшения в модели кода CRC Python IDE
 - реализовано 40 инструментов для статического анализа кода (Syntax Correctness, Reference Resolve, Expected Type, Unreachable Code и др.)
 - реализовано 12 инструментов для исправления кода (Install Package, Import Reference, Create Reference, Add Qualifier и др.)
 - реализовано 8 инструментов для рефакторинга кода (Extract, Inline, Rename, Optimize Imports и др.)
 - реализовано 3 информационных ассистента для пользователей (Hover, Signature Help и Inlay Hints)
 - реализовано 4 информационных ассистента для ИИ-агентов (Problems MCP, Hover MCP, Signature Help MCP и Rename MCP)

Заключение

- Выполнена модернизация тестовой инфраструктуры CRC Python IDE (добавлена поддержка конфигурируемых тестовых проектов с фиксацией ожидаемых результатов для каждой группы инструментов), создано 3000 модульных тестов и 5 бенчмарков для регрессионного тестирования подсистемы, а в качестве комплексного тестирования настроены автотесты с помощью фреймворка Selenium
- Проведена апробация подсистемы на основе отзывов разработчиков и пользователей CRC Python IDE, что способствовало созданию 3 новых инструментов (Version Compatibility, Add Superclass Constructor Call и Optimize Imports) и исправлению 10 проблем (ложно-положительные/отрицательные срабатывания и др.)

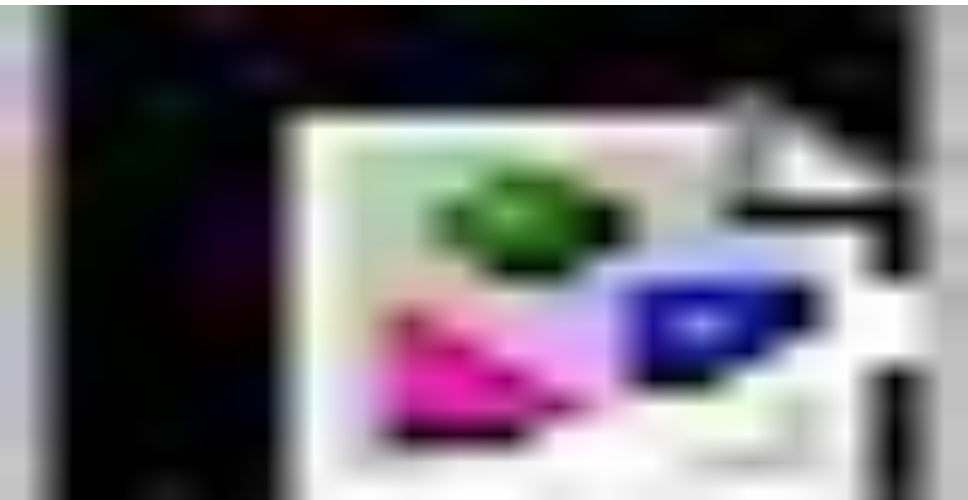
Демонстрация: анализ и модификация кода

- Пример: исправление отсутствующего вызова конструктора суперкласса с подстановкой подходящих аргументов и подсказкой сигнатуры вызова функции



Демонстрация: информационные ассистенты

- Пример: исправление синтаксической ошибки в чате с ИИ-агентом, получившим контекст проблемы



Демонстрация: мониторинг

- Пример: метрики модели кода и реализованной подсистемы для различных проектов в Grafana

